# Analysis of Noise in Diffusion Models for Image Synthesis

Austin Hartley
Clemson University
Clemson, South Carolina
aahartl@clemson.edu

## Abstract

*In the field of image synthesis, diffusion models have been proven to create high-quality images. This created a surge of research attempting to improve the training process. Most of these new studies have solely used uncorrelated Gaussian white noise at all points in training, leaving the effects of correlated noise under-researched. This paper analyzes the state-of-the-art time-varying diffusion model's use of blue noise and expands upon their research to add more insights to the relationship between the denoising process of diffusion models and the type of noise used on image results.*

## 1. Introduction

In machine learning, diffusion models have shown to be capable of creating superior results in image synthesis compared to other machine learning methods, such as Variational Autoencoders (VAEs) [5] and Generative Adversarial Networks (GANs) [2]. These results come at the expense of training time, as diffusion models are often more time-consuming to train with its iterative nature compared to GANs, where the generator and discriminator can be trained simultaneously.

Diffusion models are regularly classified as having two processes — the forward process and the backward process. In the forward process, noise is gradually added to an image iteratively until it consists of nothing but noise. In the backward process, that noised up image is then denoised with the aim of trying to reconstruct the original image. The model trains a neural network by predicting how to denoise the image at a given step until its predictions become viable. The backward process begins to reconstruct coarse shapes and structures and then slowly adds in the finer details over time, indicating that it goes from low frequency components to high frequency components. When the model is successfully trained, it is ready to generate completely new images based on any noise pattern it is given. There has not been much research comparing the behavior of the backward process or the relationship between the type of noise used in the forward and backward processes.

Most machine learning methods predominantly use uncorrelated Gaussian white noise, as it spans all frequencies evenly. This noise is very simple to generate and does not require a lot of computational power. With almost all research utilizing it, there has not been a need to look for a better alternative over improving the existing diffusion models' performance and stability.

In 2024, Huang et al. introduced the paper "Blue noise for diffusion models," [4] creating a new time-varying noise diffusion model. This allowed them to prove that blue noise, a type of correlated noise that minimizes low frequency components and leaves behind mostly high frequency components, created better results, indicating a relationship between noise and the backward process. In this paper, experiments are conducted to test the effects of other types of noise as well as the effectivity of improving image results by emphasizing low frequency components in the beginning of the backward process.

## 2. Related Work

The time-varying noise model is based off the deterministic diffusion model Iterative $\alpha$ -(de)blending (IADB) [3], but is model-agnostic and can be added to other diffusion models, such as the Deterministic Diffusion Implicit Model(DDIM) [7]. The aim of both of these deterministic models is to increase performance in diffusion models, and they were used in the comparison of results for the time-varying noise model. IADB was introduced to simplify the complex math involved in diffusion models, lowering the barrier to entry. The authors believed others could improve upon diffusion models without having graduate-level math backgrounds. Instead of trying to predict the noise at a given step in the regular diffusion process, IADB introduces an $\alpha$ parameter, which is used to blend noise in the forward process and deblend the noise in the backward process, such that the neural network is only learning what the value of $\alpha$ is.

Blue noise was utilized in the time-varying noise model because of its extensive use in computer graphics, where

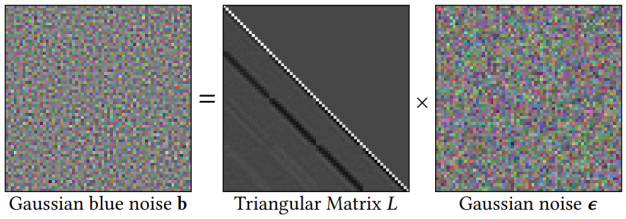


Figure 1. Blue noise mask generated on the fly using lower triangular matrix L and a Gaussian white noise vector.[4]

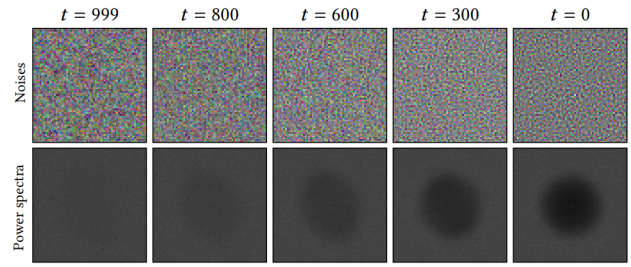


Figure 2. White noise being linearly interpolated to blue noise. Starts at 100% uncorrelated Gaussian white noise and ends at 100% blue noise. Top row shows the noise represented as RGB and bottom row shows the power spectral density.[4]

it has been in multiple denoising examples such as dithering and physically-based rendering [8]. Blue noise masks can be computationally unfriendly to generate on the fly, but through the work of previous research, [4, Huang et al.] came up with a method to do so.

## 3. Time-Varying Noise Model

The time-varying noise model starts with completely uncorrelated Gaussian white noise and slowly blends into completely Gaussian blue noise through linear interpolation, which can be seen in Figure 2.

### 3.1. Blue Noise Masks

The method to create blue noise masks on the fly involves pre-generating high quality blue noise masks [8] and finding their covariance matrices. Those covariance matrices can then be averaged to create a blue correlation matrix, $\Sigma$. A lower triangle matrix, L, can be computed from $\Sigma$, through Cholesky decomposition. The blue noise mask, **b** is denoted by the following equation:

$$\mathbf{b} = L\epsilon \tag{1}$$

where $\epsilon \sim \mathcal{N}(0, I)$ is a unit-variance Gaussian distribution. A visual representation can be seen in Figure 1. Computing L can become expensive and therefore larger masks can be created through tiling and padding smaller ones together. This will create seams that are not noticeable and do not impair the output of the results while also increasing performance.

### 3.2. White to Blue

White noise is linearly interpolated with blue noise over time, such that it starts off as only white noise and ends with being solely blue noise. This can be represented with a time-varying L, where a simple linear interpolation blend is denoted by:

$$L_t = \gamma_t L_w + (1 - \gamma_t) L_b \tag{2}$$

where $\gamma_t$ is the blending coefficient.

### 3.3. Noise Scheduling

IADB commonly uses a simple linear schedule for the value of $\alpha_t$ that is used in the forward and backward process, such as t/T, where t is the time step and T is the number of iterations.

To get the value of $\gamma_t$ for the noise interpolation, it was found that better results can be obtained by using a sigmoid function, such as:

$$sigmoid\left(\frac{\text{start} + (\text{end} - \text{start}) \cdot \frac{t}{T}}{\tau}\right) \tag{3}$$

where $sigmoid(x) = \frac{1}{1+e^{-x}}$ and $t$ is the time step. [4, Huang et al.] ran experiments to find the best values for the parameters, resulting in the start and end variables being capped between zero and three and $\tau$ being based off the image resolution. The authors note there is room for improvement in this area, allowing for better blending in noises which should create better image results.

### 3.4. Expanding IADB

IADB's original equations for the forward process, backward process, and loss function is denoted by the following equations, respectively:

$$x_t = \alpha_t \epsilon + (1 - \alpha_t) x_0 \tag{4}$$

$$x_{t-1} = x_t + (\alpha_t - \alpha_{t-1}) f_\theta(x_t, t) \tag{5}$$

$$L_{\text{IADB}} = \sum_t \left(f_\theta(x_t, t) - (x_0 - \epsilon)\right)^2 \tag{6}$$

The first equation shows noise slowly being blended into the target image $x_0$, where $\alpha_t$ is given by the linear scheduler and $\epsilon$ is uncorrelated Gaussian white noise. In the second equation, this process is inverted to deblend the noisy image. The goal of the neural network is to learn through the loss function by comparing predictions of deblending to the source.

To update these equations for the time-varying model, the forward process is essentially the same but $\epsilon$ now needs to be combined with $L_t$ to represent for the new noise over time. The backward process updates include the additional blending term to account for the new noise over time, which updates the loss function as well. The updated equations are now:

$$x_t = \alpha_t(L_t\epsilon) + (1 - \alpha_t)x_0 \tag{7}$$

$$\begin{aligned} x_{t-1} = x_t &+ (\alpha_t - \alpha_{t-1})(x_0 - L_t\epsilon) \\ &+ (\gamma_t - \gamma_{t-1})\alpha_{t-1}(L_b\epsilon - L_w\epsilon) \end{aligned} \tag{8}$$

$$\begin{aligned} L_{\text{Ours}} = \sum_t \Big( &\left(f'_\theta(x_t, t) - (x_0 - L_t\epsilon)\right)^2 + \\ &\frac{\gamma_t - \gamma_{t-1}}{\alpha_t - \alpha_{t-1}} \left(f''_\theta(x_t, t) - \alpha_{t-1}(L_b\epsilon - L_w\epsilon)\right)^2 \Big) \end{aligned} \tag{9}$$

The neural networks $f'_\theta(x_t, t)$ and $f''_\theta(x_t, t)$ are being trained to learn $x_0 - L_t\epsilon$ and $\alpha_{t-1}(L_b\epsilon - L_w\epsilon)$, respectively. The first term is learning to deblend noise back into an image and the second term is learning what the noise is at the given time step.

### 3.5. Rectified Mapping

There is correlation among pixels, since blue noise is correlated, but this model takes advantage of correlation among batches. In a standard diffusion model, noise is mapped to the image in real-time without any correlation, which can create overlaps in the mappings. In the time-varying noise model, before the forward process begins, the noise is mapped to the image based on the shortest distance, resulting in a smoother gradient flow during training as well as consistently mapping the same type of noise to a specific image [4].

## 4. Experiments

Experiments performed in [4, Huang et al.] show the white to blue blend over time beats the regular IADB method and was often very competitive with DDIM. To prove it was the higher frequency components in the blue noise creating the finer details in the images, [4, Huang et al.] ran an experiment by comparing the blend of white to blue to white to red. Red noise is a type of correlated noise that emphasizes low frequency components, thus the white and red blend lost the finer details the white and blue produced. The white to red also created more artifacts in the image due to the lack of higher frequencies. It is now evident there is a relationship between the behavior of the backward process frequency and the type of noise used, but the paper only shows the benefits of strongly emphasizing the higher frequencies as time progresses and not how important the beginning time steps are at low frequencies.

The following experiments are performed using the AFHQ-Cat dataset [1] and are compared with the results from [4], which include the white to blue noise and solely white noise at sizes 64x64 (Figures 3 and 4) and 128x128 (Figures 12 and 13).

### 4.1. Red to Blue

This experiment involved using white noise blended with red noise for a duration where red noise is emphasized at the beginning and loses emphasis over time during the blending interpolation. For the remaining time, the experiment switched to using white noise blended with blue noise, where blue noise gains emphasis over time.

For the 64x64 dimensions, the first test used the red to white blend for the first 80% and switched to the white to blue blend for the remaining 20%, which is denoted as rtob_80-20_64 in Table 1 and seen in Figure 7. This was to further prove that going against the relationship of the backward process with the noise gives worse results than following along. The second test used the red to white blend for the first 50% and switched to the white to blue blend for the remaining 50%, which is denoted as rtob_50-50_64 in Table 1 and seen in Figure 8. This is the first test to check if emphasizing low frequency in the beginning produces better results, and the data is clear that 50% is too long and produces worse results than just using the blue noise blend from the start. The third test used the red to white blend for the first 20% and switched to the white to blue blend for the remaining 80%, which is denoted as rtob_20-80_64 in Table 1 and seen in Figure 9. This test ran better than the previous ones in terms of loss in training, but the cat looked better in the 50-50 blend test and does not compete with the blue noise blend.

For the 128x128 dimensions, the trends followed those of the 64x64. However, the 20-80 model produced a better cat image than the 50-50 as seen in Figures 14 and 15. This indicates that emphasizing lower frequencies in the beginning will produce better results even if the current implementation is still not as good as the white to blue blend. This would also indicate that a smaller emphasis is more beneficial than a strong one.

### 4.2. Red and Blue

This experiment involved using a blend of red and blue noise that is later blended with white noise, where the red and blue noise blend emphasizes red at the beginning (majority red) and emphasizes blue until towards the end (majority blue). The red and blue blend was tested with a linear interpolation and a quadratic interpolation, with both being blended linearly with the white noise.

For the 64x64 dimensions, the first test used the linear blend of red and blue noise, which is denoted as randb_linear_64 in Table 1 and seen in Figure 10. This

was to further prove that emphasizing low frequency in the beginning is beneficial. The second test used the quadratic blend of red and blue noise, which is denoted as randb_quad_64 in Table 1 and seen in Figure 11. This added evidence that overly emphasizing the low frequency at the start is not as beneficial as emphasizing high frequency at the end. In both these tests, the images were very similar and created a rainbow artifact, meaning the blending needed to be tailored towards the image dimensions.

For the 128x128 dimensions, the trends followed those of the 64x64, however this time it is clear the linear blend produced a much better cat image as seen in Figures 16 and 17, despite having higher losses. This proves that lowering the low frequency emphasis at the start increased the results, but the blending is suffering from non-tailored scheduling and cannot compete with the white and blue blend without extensive tweaking. It produced better facial structures for the cat compared to solely white noise but suffered in the eye area.

### 4.3. Perlin

Perlin noise is a type of gradient noise using pseudo-random numbers and is more closely related to a low frequency noise such as red noise [6]. It is often used for terrain generation and organic textures. This involves using white noise blended with Perlin noise for a duration where white noise is emphasized at the beginning (majority white) and Perlin noise gains emphasis over time until the end (majority Perlin noise).

For the 64x64 dimensions, multiple tests used random Perlin parameters, which is denoted as perlin3_64 and perlin6_64 in Table 1 and seen in Figures 5 and 6. This tested for unique patterns in the cat's fur not seen from the other models. The cats themselves had a few artifacts, which was expected due to the low frequency bias, and the fur patterns changed, indicating that placing noise in specific areas could enhance a certain type of style change.

### 4.4. Results

The coarse to fine (low to high frequency) behavior in the backward process has a relationship with the type of noise used. Higher frequency noise over time results in significantly increased finer details. While emphasizing the lower frequency in the beginning can have an increase in image output, it is not as strong as the high frequency emphasization. The data also shows that emphasization of low frequency components at the start should not be too strong or go on for a long duration. It is not a simple task to emphasize the lower frequency components in the beginning, as it can easily produce worse results than the simple white to blue blend. Creating tailored noise for a specific type of image generation could be beneficial, but may also be limited by the dataset used, such as the face pose of the models if noise is being placed in particular places. Blending noises together increases the intricacies for training, but the tailoring of the blended noise is not a straightforward process.

| Model Name | Min Loss | Max Loss | Final Loss | Avg Loss |
|---|---|---|---|---|
| perlin6_64 | 23.671 | 903.214 | 59.900 | 64.813 |
| gwn2bn_64 | 57.160 | 1004.452 | 85.667 | 81.433 |
| perlin3_64 | 33.535 | 1090.674 | 59.348 | 84.942 |
| rtob_20-80_64 | 104.524 | 1240.373 | 120.946 | 131.349 |
| randb_quad_64 | 77.368 | 1136.757 | 140.890 | 132.746 |
| randb_linear_64 | 86.908 | 1195.852 | 154.057 | 146.570 |
| rtob_50-50_64 | 106.341 | 1249.806 | 140.891 | 152.078 |
| rtob_80-20_64 | 123.847 | 1330.491 | 174.366 | 189.349 |
| gwn_64 | 307.247 | 4229.163 | 392.268 | 437.554 |

Table 1. Losses for different models

| Model Name | Min Loss | Max Loss | Final Loss | Avg Loss |
|---|---|---|---|---|
| randb_quad_128 | 223.772 | 4492.168 | 269.177 | 324.505 |
| randb_linear_128 | 232.771 | 4519.050 | 285.998 | 348.193 |
| rtob_20-80_128 | 242.400 | 4556.963 | 302.131 | 366.362 |
| rtob_50-50_128 | 241.192 | 4549.386 | 306.109 | 395.691 |

Table 2. Losses for different models

## 5. Conclusion

The data showing that an overly strong emphasis on low frequency at the beginning is not beneficial, may imply that spending extra time to tailor a noise blend or noise scheduler for a task is not worth it. But it should be interpreted as more information gained on exactly how to blend noise for a better image results, as better blending improves training and has the ability to outperform the current white to blue noise blend. There is also more to be gained by researching how the strategic placement of noise in certain areas can create stylistic changes, as noise is a viable step in improving diffusion models.

## References


[1] Yunjey Choi, Youngjung Uh, Jaejun Yoo, and Jung-Woo Ha. Stargan v2: Diverse image synthesis for multiple domains. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8188–8197, 2020. 3

[2] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021. 1

[3] Eric Heitz, Laurent Belcour, and Thomas Chambon. Iterative $\alpha$-(de)blending: A minimalist deterministic diffusion model. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–8, 2023. 1

[4] Xingchang Huang, Corentin Salaün, Cristina Vasconcelos, Christian Theobalt, Cengiz Öztireli, and Gurprit Singh. Blue noise for diffusion models, 2024. 1, 2, 3

[5] Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019. 1

[6] K. Perlin. An image synthesizer. *SIGGRAPH*, pages 287–296, 1985. 4

[7] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *9th International Conference on Learning Representations, ICLR 2021*, Virtual Event, Austria, 2021. OpenReview.net. 1

[8] Robert Ulichney. Void-and-cluster method for dither array generation. In *Electronic Imaging*, 1993. 2

Figure 3. Gaussian White Noise 64x64

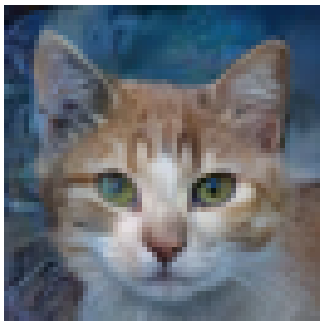

Figure 4. Gaussian Blue Noise 64x64

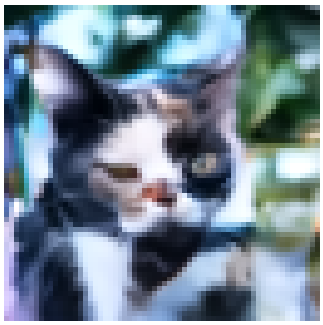

Figure 5. Perlin Noise 64x64

Figure 6. Perlin Noise 64x64

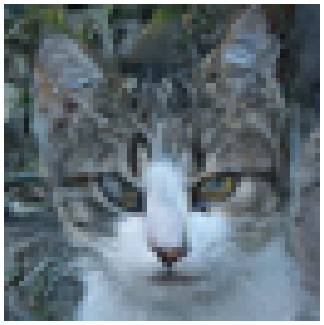

Figure 7. RtoB(80-20) Noise 64x64

Figure 8. RtoB(50-50) Noise 64x64

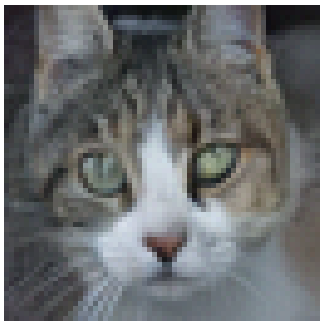

Figure 9. RtoB(20-80) Noise 64x64

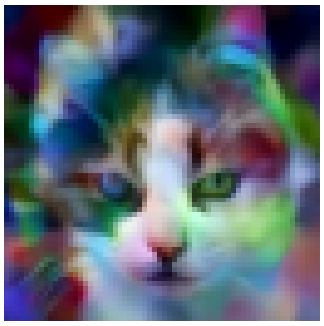

Figure 10. RandB(Linear) Noise 64x64

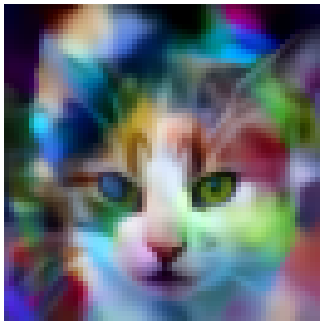

Figure 11. RandB(Quadratic) Noise 64x64

Figure 12. Gaussian White Noise 128x128

Figure 13. Gaussian Blue Noise 128x128

Figure 14. RtoB(20-80) 128x128

Figure 15. RtoB(50-50) 128x128

Figure 16. RandB(linear) 128x128

Figure 17. RandB(quadratic) 128x128